ESP32 WiFi Programming – Guide

1. Objective

Learn to build ESP32 WiFi connection code from scratch using ESP-IDF.

Understand initialization, configuration, event handling, and debugging.

2. Required Header Files

```
#include <stdio.h>

#include "freertos/FreeRTOS.h"

#include "freertos/task.h"

#include "esp_wifi.h"

#include "esp_event.h"

#include "esp_netif.h"

#include "nvs_flash.h"
```

Each header provides:

- FreeRTOS: task control

- esp_wifi: WiFi APIs

- esp_event: event system

- esp_netif: TCP/IP stack

- nvs_flash: storage

3. Step-by-Step Code Construction

Step 1: Initialize NVS

```
nvs_flash_init();
```

Step 2: Initialize Network

```
esp_netif_init();
```

Step 3: Create Event Loop

```
esp_event_loop_create_default();
```

Step 4: Create WiFi Station

```
esp_netif_create_default_wifi_sta();
```

Step 5: Initialize WiFi Driver

```
wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
esp_wifi_init(&cfg);
```

Step 6: Register Event Handler

```
esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, handler, NULL);
esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, handler, NULL);
```

Step 7: Configure WiFi

```
wifi_config_t config = {
   .sta = {
      .ssid = "SSID",
      .password = "PASSWORD",
   },
};
```

Step 8: Set Mode

esp_wifi_set_mode(WIFI_MODE_STA);

Step 9: Apply Config

esp_wifi_set_config(WIFI_IF_STA, &config);

Step 10: Start WiFi

esp_wifi_start();

4. Event Handling Logic

Events drive behavior.

WIFI_EVENT_STA_START:

→ Call esp_wifi_connect()

WIFI_EVENT_STA_DISCONNECTED:

→ Print reason

→ Retry connection

IP_EVENT_STA_GOT_IP:

→ Print IP

→ Connection successful

5. Flowchart

app_main()

↓

esp_wifi_start()

↓

```
WIFI_EVENT_STA_START
 ↓
wifi_event_handler()
 ↓
esp_wifi_connect()
 ↓
  ├── FAIL → WIFI_EVENT_STA_DISCONNECTED → Retry
  └── SUCCESS → IP_EVENT_STA_GOT_IP → Print IP
```

6. Important Macros

WIFI_EVENT_STA_START

WIFI_EVENT_STA_DISCONNECTED

IP_EVENT_STA_GOT_IP

WIFI_MODE_STA

WIFI_IF_STA

IPSTR

IP2STR()

ESP_EVENT_ANY_ID

7. Common Errors

Wrong password → AUTH_FAIL

No AP → NO_AP_FOUND

No handler → No response

8. Full Minimal Code Template

```
void app_main(void)
{
  nvs_flash_init();
```

```
    esp_netif_init();

    esp_event_loop_create_default();

    esp_netif_create_default_wifi_sta();


    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();

    esp_wifi_init(&cfg);


    esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, wifi_event_handler,
NULL);

    esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, wifi_event_handler,
NULL);


    wifi_config_t wifi_config = {

        .sta = {

            .ssid = "SSID",

            .password = "PASSWORD",

        },

    };


    esp_wifi_set_mode(WIFI_MODE_STA);

    esp_wifi_set_config(WIFI_IF_STA, &wifi_config);

    esp_wifi_start();

}
```

9. Key Takeaway

ESP32 WiFi connection is event-driven.

Main initializes system.

Events control execution.

You respond to events using handler.